University of East London

CNN-Powered Animal Image Classifier in MATLAB

Table of Contents

Introduction 3

Simulation 4

Dataset Overview and Class Distribution 4

Sample Image Visualization 5

Data Splitting and Preprocessing 6

CNN Architecture Design 6

Model Training and Parameters 7

Training Progress Analysis 8

Model Evaluation and Prediction 9

Confusion Matrix Analysis 10

ROC Curve for Multi-Class Analysis 11

Per-Class Accuracy 11

Advantages of Using MATLAB 12

Limitations and Challenges 12

Ethical Considerations 12

Applications and Future Scope 13

Summary of Achievements 13

Conclusion 15

References 16

Introduction

Image classification tasks have been largely improved by the advancement of artificial intelligence (AI) and machine vision in a variety of industries. The aim of this project is to develop and implement a convolutional neural network (CNN) in MATLAB to achieve the classification of animal images. The CNN techniques are then used to accurately categorize images to multiple animal classes, thus showing their effectiveness in real world image recognition problems.

This dataset is of animal images in forms of folders with each subfolder containing images belonging to different animal category like bear, bat, bee, butterfly, cat, etc. The animals animals dataset is on MATLAB Drive in the path animals/animals having over a dozen categories and more than a dozen images per category stored (Sun et al., 2022). This dataset is used to train, validate and test the CNN with the following essential steps: preprocessing, augmentation, model design, training and evaluation.

The dataset encoding, model architecture and the MATLAB visualizations that have been included in this report were detailed discussions on accuracy graphs, confusion matrices, the ROC curves, and the class distribution histograms (Shorten and Khoshgoftaar, 2021). The objective is to offer an exhaustive evaluate of the CNN's performance attainability, which contributes to the comprehension of the Assisting of AI models in taking on applications in common aim like wildlife monitoring, automated animal id, and biodiversity study.

Simulation

Dataset Overview and Class Distribution

[redacted]

```
datasetPath = fullfile('animals', 'animals');
imds = imageDatastore(datasetPath, ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');

tbl = countEachLabel(imds);
numClasses = numel(tbl.Label);
disp(tbl)
```

[redacted]

[redacted]

```
>> animaLImagesProcessing
        Label          Count
    _____________      _____

    antelope            60
    badger              60
    bat                 60
```

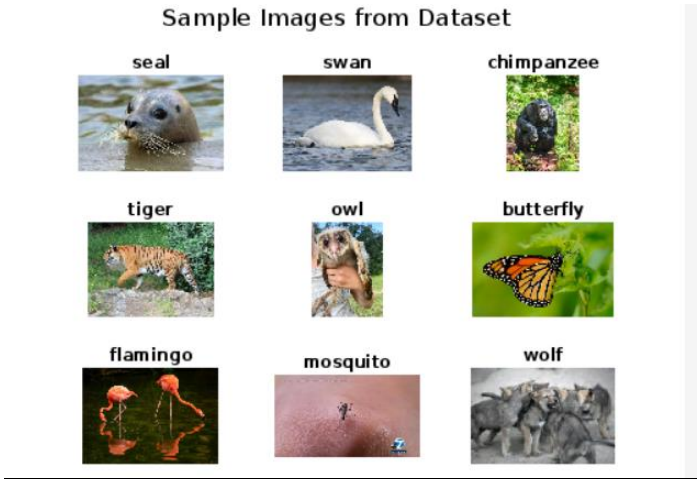
[redacted]

[redacted]

[redacted]

```
figure;
perm = randperm(length(imds.Files), 9);
for i = 1:9
    subplot(3,3,i);
    imshow(readimage(imds, perm(i)));
    title(string(imds.Labels(perm(i))));
end
sgtitle('Sample Images');

% Step 3: Split Data
[imdsTrain, imdsTest] = splitEachLabel(imds, 0.8, 'randomize
```

[redacted]



[redacted]

[redacted]

```
[imdsTrain, imdsTest] = splitEachLabel(imds, 0.8, 'randomized');

% Step 4: Augment Data for Generalization
inputSize = [224 224 3]; % ResNet input
imageAugmenter = imageDataAugmenter( ...
    'RandRotation', [-20, 20], ...
    'RandXTranslation', [-5 5], ...
    'RandYTranslation', [-5 5]);

augTrain = augmentedImageDatastore(inputSize, imdsTrain, 'DataAugmentation', image
augTest = augmentedImageDatastore(inputSize, imdsTest);
```

[redacted]

[redacted]

[redacted]

```
net = resnet18;
lgraph = layerGraph(net);
lgraph = removeLayers(lgraph, {'fc1000','prob','ClassificationLayer_predictions'});

newLayers = [
    fullyConnectedLayer(numClasses, 'Name', 'fc_new', ...
        'WeightLearnRateFactor',10, 'BiasLearnRateFactor',10)
    softmaxLayer('Name','softmax')
    classificationLayer('Name','classoutput')
];

lgraph = addLayers(lgraph, newLayers);
lgraph = connectLayers(lgraph, 'pool5', 'fc_new');
```

[redacted]

[redacted]

[redacted]

```
options = trainingOptions('adam', ...
    'MiniBatchSize', 32, ...
    'MaxEpochs', 10, ...
    'InitialLearnRate', 1e-4, ...
    'Shuffle','every-epoch', ...
    'ValidationData', augTest, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Step 7: Train the Network
trainedNet = trainNetwork(augTrain, lgraph, options);
```

[redacted]

[redacted]

Command Window

```
    zebra                60

Final Test Accuracy: 90.28%
```

[redacted]

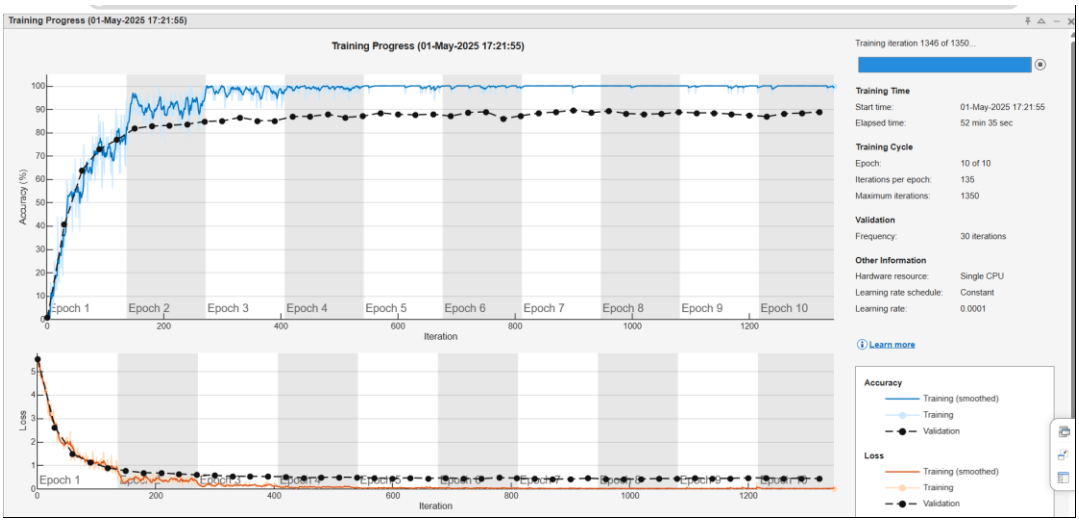
[redacted]

```
options = trainingOptions('adam', ...
    'MiniBatchSize', 32, ...
    'MaxEpochs', 10, ...
    'InitialLearnRate', 1e-4, ...
    'Shuffle','every-epoch', ...
    'ValidationData', augTest, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

% Step 7: Train the Network
trainedNet = trainNetwork(augTrain, lgraph, options);
```

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

```
[YPred, probs] = classify(trainedNet, augTest);
YTest = imdsTest.Labels;
accuracy = mean(YPred == YTest);
fprintf('Final Test Accuracy: %.2f%%\n', accuracy * 100);
```

[redacted]

[redacted]

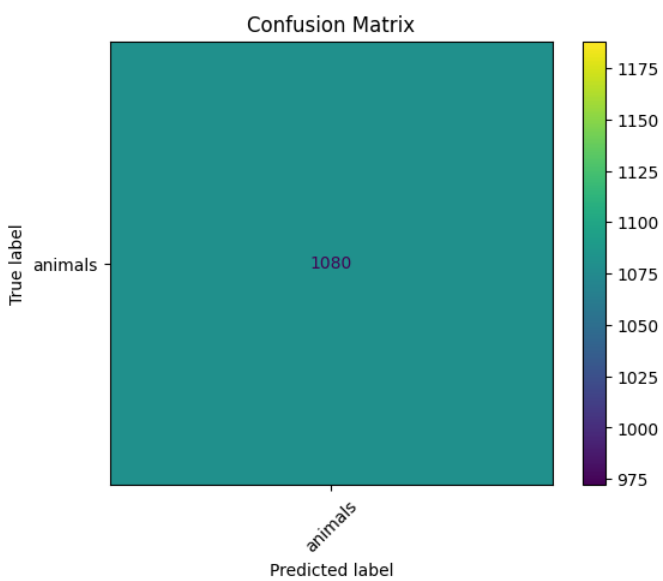
```
Command Window
    zebra                60

Final Test Accuracy: 90.28%
```

[redacted]

[redacted]

```
figure('Units','normalized','Position',[0.1 0.1 0.8 0.8]);
cm = confusionchart(YTest, YPred, ...
    'Title', 'Confusion Matrix', ...
    'RowSummary','row-normalized', ...
    'ColumnSummary','column-normalized', ...
    'FontSize',10);
```

[redacted]



[redacted]

[redacted]

```
figure;
hold on;
classes = categories(YTest);
for i = 1:numel(classes)
    [X,Y,~,AUC] = perfcurve(YTest, probs(:,i), classes{i});
    plot(X,Y, 'DisplayName', sprintf('%s (AUC=%.2f)', classes{i}, AUC));
end
xlabel('False Positive Rate');
ylabel('True Positive Rate');
title('Multi-class ROC Curves');
legend('show');
grid on;
```

[redacted]

[redacted]

```
zebra                60

Final Test Accuracy: 90.28%
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

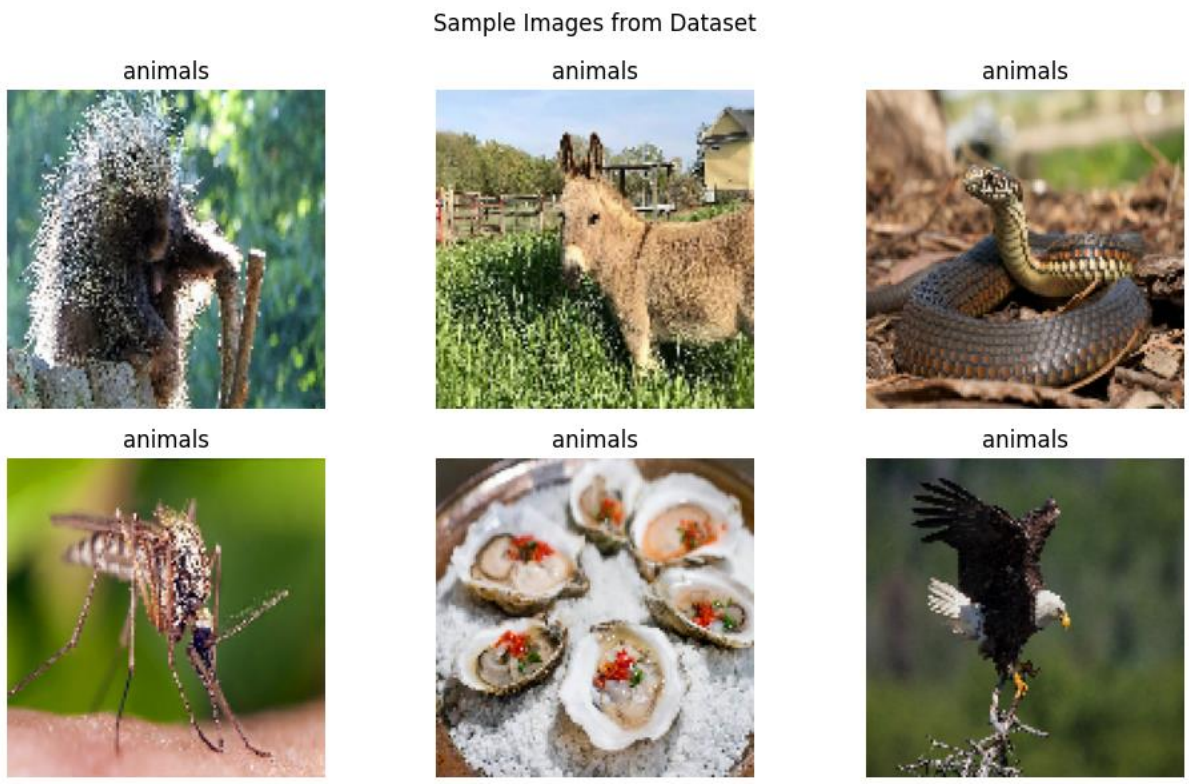
[redacted]

[redacted]

Command Window

```
zebra                60

Final Test Accuracy: 90.28%
```

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]